

The Five-Fold and Four-Fold Rule/Fact Layer in OMI

A Whitepaper on 5! Packet Roots, 4! Selector Surfaces, and the Derivation of RULES.omi into FACTS.omi

Status: Conceptual Whitepaper / v1.0.0-RC1 Alignment Draft

Scope: OMI rule/fact derivation, five-fold packet roots, four-fold selector surfaces, 240-state bridge, replay-ring decomposition, semantic routing, and canvas projection

Core Claim: The 5-fold layer gives OMI its rooted packet identity, while the 4-fold layer gives OMI its projectable selector/channel structure. Together, they explain how abstract rules become grounded facts without leaving the 128-bit OMI wire frame.

Abstract

OMI is an address-centered protocol system. Its core object is the 128-bit OMI pointer:

omi-S0-S1-S2-S3-S4-S5-S6-S7/prefix

Each pointer is not merely an identifier. It is a reference surface that can bind rules, facts, tests, replay receipts, semantic triples, and visible projections.

This paper explains the special role of the **five-fold** and **four-fold** layers in that system.

The five-fold layer corresponds to:

$5! = 120$

It is the rooted packet core, the 5-cell simplex layer, and the hidden structural source inside the 240-state bridge:

$240 / 2 = 120 = 5!$

The four-fold layer corresponds to:

$4! = 24$

It is the selector/channel layer: the 24-cell face, the FS/GS/RS/US channel plane, the 4-bit nibble control surface, and the place where OMI rules become projectable facts.

Together:

5-fold = packet root / semantic simplex / hidden core
4-fold = selector surface / channel plane / visible projection hinge

The central thesis is:

The five-fold layer roots the packet.
The four-fold layer exposes the packet.
Rules live as five-fold obligations.
Facts become visible through four-fold projections.

1. Why Five-Fold and Four-Fold Matter

OMI already has a factorial tower:

1!, 2!, 3!, 4!, 5!, 6!, 7!, 8!, 9!, 10!, 11!, 12!

But not every layer plays the same role.

The lower layers are structural:

1! = identity
2! = complement / binary fold
3! = line / S-P-O seed
4! = selector / channel surface
5! = packet root / simplex core

The higher layers are envelopes:

6! = semantic sweep
7! = Fano replay ring
8! = 8-segment physical frame universe
9! = route/gate envelope
10! = declaration envelope
11! = witness/proof envelope
12! = clock/provider envelope

The 5-fold and 4-fold layers are the most important hinge point because they are where the system changes from:

abstract packet identity

to:

addressable projected state

That is why 5! and 4! should be treated as the **rule/fact transition core**.

2. The Five-Fold Layer

The five-fold layer is:

5! = 120

In OMI, this is the **packet root**.

It represents the smallest complete packet universe before orientation, semantic sweep, Fano replay, route envelopes, or clock/provider phases are applied.

The five-fold layer can be understood in five equivalent ways:

1. 5! packet core
2. 5-cell simplex
3. five semantic cells
4. rooted hidden factor inside 240
5. rule obligation core

2.1 Five-Fold as 5! Packet Core

The identity:

5! = 120

means there are 120 permutations of five distinct positions.

In OMI, those five positions are the minimal packet factors.

A useful five-fold packet model is:

1. subject seed
2. predicate seed
3. object seed
4. context/witness seed
5. closure/check seed

So a five-fold packet is not just a data item. It is a complete semantic unit:

subject + predicate + object + context + closure

This is the rooted packet identity.

2.2 Five-Fold as 5-Cell

Geometrically, the five-fold layer maps to the 5-cell, the 4D simplex.

A 5-cell has five tetrahedral cells.

OMI uses this as a model for a minimal complete semantic enclosure:

five vertices / five cells / five roles

The packet is “inside” the 5-cell. Its meaning is stable because all five roles can be permuted without leaving the simplex boundary.

Thus:

$5! =$ all role-orderings of the 5-cell packet core

2.3 Five-Fold as Hidden Root in the 240-State Bridge

The 240-state bridge is:

$240 = 16 \times 16 - 16$
 $240 = 2 \times 5!$

Therefore:

$$5! = 240 / 2$$

This means the 5-fold packet core is not erased when OMI moves to 240. It is doubled by orientation:

$$5! \text{ packet core} \times 2 \text{ orientations} = 240 \text{ oriented packet states}$$

The $\times 2$ is the complement/chirality layer:

direct / folded
valid / failure
presence / absence
source / shadow
left / right

So:

$$240 = \text{oriented five-fold packet space}$$

The five-fold core is rooted, while 240 is its oriented carrier.

3. The Four-Fold Layer

The four-fold layer is:

$$4! = 24$$

In OMI, this is the **selector surface**.

It is where packet identity becomes projectable.

The four-fold layer can be understood in five equivalent ways:

1. 4! selector space
2. 24-cell face layer
3. FS/GS/RS/US channel plane

- 4. 4-bit nibble surface
- 5. fact projection hinge

3.1 Four-Fold as 4! Selector Space

The identity:

$$4! = 24$$

means there are 24 permutations of four distinct positions.

In OMI, those four positions are the basic channel faces:

FS
GS
RS
US

These can be read as:

FS = File / Frame / Storage
GS = Group / Graph / Global
RS = Record / Relation / Routing
US = Unit / Text / Terminal

A four-fold fact is a fact that can be projected into one or more of these channels.

3.2 Four-Fold as 24-Cell Face Layer

Geometrically, $4! = 24$ maps to the 24-cell or 24-position selector lattice.

The 24-cell is important in OMI because it gives a self-dual projection layer:

inside/outside
rule/fact
source/shadow
valid/failure

So $4! = 24$ is the first layer where the rooted 5-fold packet can become visibly selectable.

In short:

5-fold packet = contained meaning
4-fold selector = exposed face

3.3 Four-Fold as Nibble Logic

The four-fold layer also maps to nibble logic.

A nibble has:

4 bits
 $2^4 = 16$ possible values

The active nibble surface is:

$2^4 - 1 = 15$

This is why 4-fold logic connects to the 240-state bridge:

$15 \times 16 = 240$

Four bits give the carrier. Fifteen active states give the nonzero surface. Together with a full 16-state carrier rail, they form the active byte surface.

Therefore:

4-fold selector logic produces the byte-level surface that carries the 5-fold packet root.

4. Rules and Facts

OMI separates rules from facts.

A rule is normative:

<omi-pointer> MUST <invariant-name>

A fact is grounded:

```
<omi-pointer> FACT <applied-fact-name>
```

A rule says:

```
This must be true for conformance.
```

A fact says:

```
This is grounded in the current system.
```

The provider contract is:

```
No address without a rule.  
No rule without a test.  
No test without a replay path.  
No replay path without visible state.
```

The five-fold/four-fold distinction clarifies this:

```
Five-fold rules define packet obligations.  
Four-fold facts expose grounded packet state through selectable channels.
```

5. Five-Fold Rules

A five-fold rule is a rule that protects the rooted packet core.

It constrains:

```
semantic identity  
packet closure  
payload coherence  
simplex role order  
source/fold relation
```

A five-fold rule asks:


```
Is the packet complete?
Is the packet coherent?
Does the packet preserve its root identity?
Can the packet survive orientation/complement?
Can the packet enter the 240 bridge lawfully?
```

5.1 Five-Fold Rule Template

```
# [Rule 0x___]: Five-Fold Packet Root
#   This rule protects the 5! packet core before orientation, projection,
#   semantic sweep, or provider clock phase is applied.
omi-....-.....-.....-.....-.....-.....-...../prefix MUST preserve-five-fold-
packet-root
```

5.2 Five-Fold Rule Fields

A five-fold rule should define:

1. packet subject seed
2. packet predicate seed
3. packet object seed
4. packet witness/context seed
5. packet closure/check seed

In a 128-bit OMI frame, this may be distributed across:

```
S0 = LL rail / opener
S2 = N payload / subject seed
S3 = predicate/operator rail
S4 = relation/operator rail
S5 = M payload / object/witness seed
S7 = closure rail
```

The exact mapping can vary, but the rule must preserve the five-fold packet identity.

5.3 Five-Fold Rule Examples

```
omi-0000-0000-0000-0000-0000-0000-0005-0001/128 MUST preserve-five-fold-packet-
root
omi-0000-0000-0000-0000-0000-0000-0005-0002/128 MUST bind-five-fold-packet-
to-240-bridge
```

```
omi-0000-0000-0000-0000-0000-0000-0005-0003/128 MUST preserve-five-cell-semantic-closure
omi-0000-0000-0000-0000-0000-0000-0005-0004/128 MUST reject-unclosed-five-fold-packets
omi-0000-0000-0000-0000-0000-0000-0005-0005/128 MUST expose-five-fold-packet-to-four-fold-selector
```

6. Four-Fold Rules

A four-fold rule is a rule that controls projection, selection, or channel routing.

It constrains:

```
FS/GS/RS/US routing
4-bit selectors
24-cell face projection
canvas/CSSOM visibility
fact-channel exposure
```

A four-fold rule asks:

```
Which channel receives the packet?
Which selector face is active?
Which projection is visible?
Which channel owns failure behavior?
Which surface shadows the source rule?
```

6.1 Four-Fold Rule Template

```
# [Rule 0x___]: Four-Fold Selector Projection
#   This rule exposes a grounded packet through FS/GS/RS/US or an equivalent
#   four-channel selector surface.
omi-....-....-....-....-....-....-....-..../prefix MUST project-four-fold-selector-surface
```

6.2 Four-Fold Selector Channels

The canonical four channels are:

```
FS = file / frame / storage
GS = group / graph / global
RS = relation / routing / record
US = unit / text / terminal
```

A four-fold selector should decide where state is visible.

Example:

```
FS → file/blob/frame egress
GS → graph/group memory
RS → relation/link routing
US → unit/text/symbol stream
```

6.3 Four-Fold Rule Examples

```
omi-0000-0000-0000-0000-0000-0000-0004-0001/128 MUST project-four-fold-selector-
surface
omi-0000-0000-0000-0000-0000-0000-0004-0002/128 MUST route-state-through-fs-gs-
rs-us
omi-0000-0000-0000-0000-0000-0000-0004-0003/128 MUST encode-selector-as-four-
bit-nibble
omi-0000-0000-0000-0000-0000-0000-0004-0004/128 MUST expose-fact-through-24-
cell-face
omi-0000-0000-0000-0000-0000-0000-0004-0005/128 MUST reject-orphan-four-fold-
projections
```

7. Five-Fold Facts

A five-fold fact is a grounded declaration that says the packet root exists or has been verified.

It should answer:

```
Which five-fold packet exists?
Which source rule grounds it?
Which test proves it?
Which replay path stores it?
Which four-fold selector exposes it?
```

7.1 Five-Fold Fact Template

```
# [Derived from Rule 0x___]
# Rule: preserve-five-fold-packet-root
# Transition: unverified -> packet-root-verified
# Bridge: 5! = 120 = 240 / 2
omi-....-....-....-....-....-....-....-..../prefix FACT five-fold-packet-root-verified
```

7.2 Five-Fold Fact Examples

```
omi-0000-0000-0000-0000-0000-0000-0005-1001/128 FACT five-fold-packet-root-verified
omi-0000-0000-0000-0000-0000-0000-0005-1002/128 FACT five-fold-packet-bound-to-240-bridge
omi-0000-0000-0000-0000-0000-0000-0005-1003/128 FACT five-cell-semantic-closure-active
omi-0000-0000-0000-0000-0000-0000-0005-1004/128 FACT five-fold-packet-replayable
omi-0000-0000-0000-0000-0000-0000-0005-1005/128 FACT five-fold-packet-ready-for-four-fold-projection
```

8. Four-Fold Facts

A four-fold fact is a grounded declaration that says a packet has become visible through a selector/channel surface.

It should answer:

```
Which channel is active?
Which selector is used?
Which canvas/CSSOM surface displays it?
Which shadow address derives from the source?
Which failure behavior applies?
```

8.1 Four-Fold Fact Template

```
# [Derived from Rule 0x___]
# Rule: project-four-fold-selector-surface
# Transition: packet-root-verified -> selector-projected
```

```
# Selector: FS/GS/RS/US
omi-....-.....-.....-.....-.....-.....-.....-...../prefix FACT four-fold-selector-
projection-active
```

8.2 Four-Fold Fact Examples

```
omi-0000-0000-0000-0000-0000-0000-0004-1001/128 FACT fs-channel-projection-
active
omi-0000-0000-0000-0000-0000-0000-0004-1002/128 FACT gs-channel-projection-
active
omi-0000-0000-0000-0000-0000-0000-0004-1003/128 FACT rs-channel-projection-
active
omi-0000-0000-0000-0000-0000-0000-0004-1004/128 FACT us-channel-projection-
active
omi-0000-0000-0000-0000-0000-0000-0004-1005/128 FACT four-fold-shadow-canvas-
projection-active
```

9. The Five-Fold to Four-Fold Transition

The transition from five-fold to four-fold is:

```
packet root → selector surface
```

or:

```
5! → 4!
```

This is not a loss of information. It is a projection.

The five-fold packet remains rooted. The four-fold selector exposes one face or channel of it.

So the transition should be written:

```
five-fold packet core
→ four-fold projection face
```

not:

```
five-fold packet destroyed into four-fold state
```

A grounded fact should always preserve the source packet.

9.1 A-List Transition

```
(  
  (source-rule . "preserve-five-fold-packet-root")  
  (source-fact . "five-fold-packet-root-verified")  
  (state-before . "packet-root-verified")  
  (transition . "project-onto-four-fold-selector")  
  (state-after . "selector-projected")  
  (selector-surface . "FS/GS/RS/US")  
  (fact . "four-fold-selector-projection-active")  
  (failure-behavior . "reject-orphan-four-fold-projections")  
)
```

9.2 Bitboard Interpretation

A 4-fold selector can be held in a nibble:

```
selector4 = x & 0b1111
```

But the active selector surface should avoid the null rail when needed:

```
activeSelector15 = selector4 & 0xF
```

The 240 bridge appears when this active selector is paired with a full carrier nibble:

```
local240 = active15 * carrier16
```

or:

```
local240 = (activeNibble * 16) + carrierNibble
```

with the zero rail excluded.

10. The 240 Bridge as the Meeting Point

The five-fold and four-fold layers meet at 240.

The identities are:

```
5! = 120
240 = 2 × 5!
240 = 16 × 15
240 = 16 × 16 - 16
6! = 3 × 240
7! = 7 × 3 × 240
```

Interpretation:

```
5!    = packet root
×2    = orientation/complement
240   = oriented packet surface
×3    = S-P-0 semantic sweep
6!    = semantic sweep
×7    = Fano selector
7!    = replay ring
```

Thus the five-fold packet becomes a 240-state field only after adding orientation.

The four-fold selector becomes a 240-state field by producing the active byte surface:

```
15 active nibble values × 16 carrier values = 240
```

So both meet at the same bridge:

```
five-fold side: 2 × 5! = 240
four-fold side: 15 × 16 = 240
```

This is the key result.

```
5-fold explains why 240 is semantically rooted.
4-fold explains why 240 is byte-addressable.
```

11. Canonical 5040 Slot Decomposition

The 5040 replay ring is:

$$7! = 5040$$

Using the 240 bridge:

$$5040 = 7 \times 3 \times 240$$

This gives the canonical slot formula:

$$\text{slot5040} = \text{fano7} \times 720 + \text{role3} \times 240 + \text{local240}$$

Where:

$$\begin{aligned} \text{fano7} &\in 0..6 \\ \text{role3} &\in 0..2 \\ \text{local240} &\in 0..239 \end{aligned}$$

This is where five-fold and four-fold become operational.

The `local240` may be derived from either:

five-fold packet + orientation

or:

four-fold selector + carrier

So a replay slot can be read as:

Fano selector
+ S-P-O role
+ oriented packet/selector state

12. Five-Fold and Four-Fold in the 128-Bit Frame

A practical mapping:

```
S0 = LL rail / opener / Fano selector
S1 = 0x03BF chiral delimiter
S2 = N payload / subject or packet seed
S3 = predicate/operator rail / four-fold selector high rail
S4 = relation/operator rail / four-fold selector low rail
S5 = M payload / object or witness seed
S6 = 0x039F cardinal delimiter
S7 = LLff closure / complement rail
```

Five-fold roles are mostly packet-semantic:

```
S0/S2/S3/S4/S5/S7
```

Four-fold roles are mostly selector/projective:

```
S3/S4 plus FS/GS/RS/US channel interpretation
```

The delimiters:

```
S1 = 0x03BF
S6 = 0x039F
```

hold the chiral/cardinal boundary.

13. Five-Fold Rule-to-Fact Derivation

A complete five-fold derivation:

```
RULE:
omi-0000-0000-0000-0000-0000-0000-0005-0001/128 MUST preserve-five-fold-packet-
root

DECLARATION:
(
  (rule-id . "0x5F01")
```

```

(rule-pointer . "omi-0000-0000-0000-0000-0000-0000-0005-0001/128")
(modal . "MUST")
(invariant . "preserve-five-fold-packet-root")
(axiom-family . "five-fold-packet-root")

(state-before . "unverified")
(transition . "evaluate-five-fold-packet-closure")
(state-after . "packet-root-verified")

(bitboard-mask . "five-role-packet-mask")
(bitblip-policy . "evict-unclosed-packet")

(fact-pointer . "omi-0000-0000-0000-0000-0000-0000-0005-1001/128")
(fact-name . "five-fold-packet-root-verified")

(test . "test/five-fold-packet.test.js")
(projection . "replay-ring + shadow-canvas")
(failure-behavior . "reject-token")
)

FACT:
omi-0000-0000-0000-0000-0000-0000-0005-1001/128 FACT five-fold-packet-root-
verified

```

14. Four-Fold Rule-to-Fact Derivation

A complete four-fold derivation:

```

RULE:
omi-0000-0000-0000-0000-0000-0000-0004-0001/128 MUST project-four-fold-selector-
surface

DECLARATION:
(
  (rule-id . "0x4F01")
  (rule-pointer . "omi-0000-0000-0000-0000-0000-0000-0004-0001/128")
  (modal . "MUST")
  (invariant . "project-four-fold-selector-surface")
  (axiom-family . "four-fold-selector-projection")

  (state-before . "packet-root-verified")
  (transition . "project-onto-fs-gs-rs-us")
  (state-after . "selector-projected")
)

```

```

(segment-mask . "[0000,0000,0000,0000,ffff,ffff,0000,0000]")
(bitboard-mask . "four-channel-selector-mask")
(bitblip-policy . "warn-near-miss")

(fact-pointer . "omi-0000-0000-0000-0000-0000-0000-0004-1001/128")
(fact-name . "four-fold-selector-projection-active")

(test . "test/four-fold-selector.test.js")
(projection . "cssom + json-canvas")
(failure-behavior . "reject-orphan-projection")
)

FACT:
omi-0000-0000-0000-0000-0000-0000-0004-1001/128 FACT four-fold-selector-
projection-active

```

15. Relationship to Shadow Canvas Addressing

The four-fold layer is the natural place to introduce shadow canvas addressing.

A source rule/fact should derive a canvas shadow pointer:

```

source rule/fact pointer
→ four-fold selector
→ shadow canvas address

```

Example:

```

SOURCE:
omi-0000-0000-0000-0000-0000-0000-0005-1001/128 FACT five-fold-packet-root-
verified

SHADOW:
omi-ca55-0000-0000-0000-0000-0000-0005-1001/128 FACT shadow-canvas-five-fold-
packet-root

```

The `ca55` segment marks a canvas shadow namespace.

Four-fold projection guarantees this shadow is not orphaned.

No shadow canvas state without a source rule or fact.

16. Relationship to One-Word OMI Rewrite

The one-word rewrite layer should use five-fold/four-fold structure as follows:

```
one 128-bit word
→ verify five-fold packet root
→ select four-fold projection channel
→ derive shadow canvas address
→ emit replay receipt
```

In pseudocode:

```
function rewriteOneWord(frame) {
  const packet5 = verifyFiveFoldPacket(frame);
  if (!packet5.valid) return reject("five-fold packet invalid");

  const selector4 = deriveFourFoldSelector(frame);
  const local240 = deriveLocal240(packet5, selector4);

  const slot5040 =
    packet5.fano7 * 720 +
    packet5.role3 * 240 +
    local240;

  const shadow = deriveShadowCanvasAddress(frame, selector4);
  const receipt = packReceipt(frame, slot5040, shadow);

  return {
    packet5,
    selector4,
    local240,
    slot5040,
    shadow,
    receipt,
    projection: "visible"
  };
}
```

This keeps the rewrite function rooted:

five-fold packet root
 → four-fold projection
 → 240 bridge
 → 5040 replay slot
 → visible shadow

17. Five-Fold vs Four-Fold Summary

Layer	Formula	Role	OMI Function
Five-fold	$5! = 120$	packet root	semantic closure, 5-cell core
Orientation	$\times 2$	complement/chirality	direct/folded surface
Bridge	$240 = 2 \times 5!$	oriented packet	active byte surface
Four-fold	$4! = 24$	selector surface	channel/projection face
Nibble	$2^4 = 16$	carrier	hex control surface
Active nibble	15	nonzero rail	active selector state
Bridge from 4-fold	$15 \times 16 = 240$	active byte	byte-addressable projection
Semantic sweep	$3 \times 240 = 720$	S-P-O	6! promote layer
Replay ring	$7 \times 720 = 5040$	Fano replay	7! runtime memory

The central equality is:

$$2 \times 5! = 15 \times 16 = 240$$

This says:

the five-fold packet root and the four-fold selector surface meet in the same 240-state bridge.

18. Rules to Add

18.1 Five-Fold Rule Band

```
# [Rule 0x6C]: Five-Fold Packet Root Preservation
omi-0000-0000-0000-0000-0000-0000-0005-0001/128 MUST preserve-five-fold-packet-
root

# [Rule 0x6D]: Five-Fold to 240 Bridge Binding
omi-0000-0000-0000-0000-0000-0000-0005-0002/128 MUST bind-five-fold-packet-
to-240-bridge

# [Rule 0x6E]: Five-Cell Semantic Closure
omi-0000-0000-0000-0000-0000-0000-0005-0003/128 MUST preserve-five-cell-
semantic-closure
```

18.2 Four-Fold Rule Band

```
# [Rule 0x6F]: Four-Fold Selector Projection
omi-0000-0000-0000-0000-0000-0000-0004-0001/128 MUST project-four-fold-selector-
surface

# [Rule 0x70]: FS/GS/RS/US Channel Routing
omi-0000-0000-0000-0000-0000-0000-0004-0002/128 MUST route-state-through-fs-gs-
rs-us

# [Rule 0x71]: Shadow Canvas Projection from Four-Fold Selector
omi-ca55-0000-0000-0000-0000-0000-0004-0003/128 MUST derive-shadow-canvas-from-
four-fold-selector
```

19. Facts to Add

19.1 Five-Fold Facts

```
# --- Five-Fold Packet Root Facts ---

omi-0000-0000-0000-0000-0000-0000-0005-1001/128 FACT five-fold-packet-root-
verified
omi-0000-0000-0000-0000-0000-0000-0005-1002/128 FACT five-fold-packet-bound-
to-240-bridge
```

```
omi-0000-0000-0000-0000-0000-0000-0005-1003/128 FACT five-cell-semantic-closure-active
```

19.2 Four-Fold Facts

```
# --- Four-Fold Selector Projection Facts ---

omi-0000-0000-0000-0000-0000-0000-0004-1001/128 FACT fs-channel-projection-active
omi-0000-0000-0000-0000-0000-0000-0004-1002/128 FACT gs-channel-projection-active
omi-0000-0000-0000-0000-0000-0000-0004-1003/128 FACT rs-channel-projection-active
omi-0000-0000-0000-0000-0000-0000-0004-1004/128 FACT us-channel-projection-active
omi-ca55-0000-0000-0000-0000-0000-0004-1005/128 FACT shadow-canvas-four-fold-selector-active
```

20. Tests to Add

Suggested tests:

```
test/five-fold-packet.test.js
test/four-fold-selector.test.js
test/five-four-bridge.test.js
```

Minimum assertions:

```
five-fold packet root verifies
unclosed packet rejects
five-fold packet binds to 240 via ×2
four-fold selector derives FS/GS/RS/US
four-fold selector rejects orphan projection
local240 stays in 0..239
slot5040 formula stays in 0..5039
shadow canvas pointer derives from source fact
rewrite receipt includes source, selector, local240, slot5040
```

21. Final Position

The five-fold and four-fold layers are not competing models.

They are complementary.

```
5-fold = root  
4-fold = face
```

The five-fold layer gives OMI its packet identity:

```
5! = 120
```

The four-fold layer gives OMI its selector/projection surface:

```
4! = 24
```

They meet in the 240 bridge:

```
2 × 5! = 15 × 16 = 240
```

Then the bridge expands into semantic replay:

```
3 × 240 = 6!  
7 × 3 × 240 = 7!
```

Therefore:

```
five-fold packet root  
→ orientation/complement  
→ 240 active byte bridge  
→ four-fold selector projection  
→ S-P-O semantic sweep  
→ Fano replay ring  
→ visible OMI state
```


22. One-Sentence Summary

The five-fold layer roots OMI state in the $5! = 120$ packet/5-cell core, while the four-fold layer projects that rooted state through the $4! = 24$ selector/channel surface; both meet at the 240-state bridge, where $2 \times 5! = 15 \times 16$, allowing rules to become grounded facts, facts to become canvas projections, and projections to become replayable OMI state.